
Context is the New Weight: Distilling In-Context Adaptation into Weight Updates and Comparing the Mechanism at the Attention Layer

Zizhao Hu

Department of Computer Science
University of Southern California
zizhaohu@usc.edu

Abstract

There are three obvious ways to make a language model behave according to a context C : prepend C at inference time (**in-context**); fine-tune the model to imitate the outputs it produces under C , with no C at inference (**context-simulate-FT**); or fine-tune the model on the text of C itself with a standard language-modelling loss (**context-FT**). The three interventions look superficially similar but produce very different changes in the underlying model. On Llama-3.1-8B-Instruct and five context types — structure constraint (high and low), style constraint, few-shot, and factual injection — we measure the resulting residual-stream perturbations site by site and find: context-simulate-FT recovers the in-context target both *behaviorally* (44–90% exact-match) and *mechanistically* (peak per-site cosine 0.44–0.79 in late layers, controlled against the base flow); context-FT moves more weight in absolute Frobenius norm but reproduces neither the behavior (0–13% EM) nor the mechanism (peak cosine 0.16–0.38). A no-behavior-change control further shows that the layer-1 weight-spike pattern observed in both fine-tunes is largely an optimizer-architecture artifact, not a signature of context internalisation. The three-setting comparison operationalises the in-context-learning-as-gradient-descent hypothesis on full instruction-tuned transformers, and shows that the equivalence holds for one specific recipe and breaks for the other.

1 Introduction

A language model can be adapted to behave according to a context C in several ways. The cheapest is to prepend C at inference time and let attention do the work — this is what we call **in-context** adaptation, or **Setting 1**. The most expensive is to fine-tune the model’s weights so that the same behavior is baked in. But “fine-tuning to absorb C ” is not a single recipe: at minimum, one can fine-tune on synthetic (Q, A) pairs in which A has been generated under C (**Setting 2: context-simulate-FT**), or one can fine-tune on the text of C itself with a standard language-modelling loss (**Setting 3: context-FT**). Settings 2 and 3 produce different θ_C , different ΔW , different residual-stream behavior. Both “count” as fine-tunes; both nominally “incorporate the context.” Which one, if either, ends up at the same place as the in-context target?

This question is the empirical sibling of the in-context-learning-as-gradient-descent line of work [von Oswald et al., 2023, Akyürek et al., 2023, Garg et al., 2022]. Those papers establish, in linear-attention or other restricted regimes, that the in-context forward pass is mathematically equivalent to one step of gradient descent on a meta-objective. They do not perform the explicit gradient descent on the real transformer’s weights and check whether the result matches; they characterise what Setting 1

is doing internally. We ask the explicit version: *do Settings 2 or 3, run as actual gradient-descent fine-tunes on the real model, end up at the same place in residual-stream space as Setting 1?*

For a fixed base model M (Llama-3.1-8B-Instruct) and five context types — structure constraint (high), structure constraint (low), style constraint, few-shot, factual injection — we run all three settings, then for each fine-tuned student we measure two perturbation vectors at every post-attention and post-MLP residual-stream site:

$$\begin{aligned} v_s^{\text{ctx}} &= \text{resid}_s(Q' \mid \theta, [C; Q']) - \text{resid}_s(Q' \mid \theta, Q'), \\ v_s^{\text{dw}} &= \text{resid}_s(Q' \mid \theta_C, Q') - \text{resid}_s(Q' \mid \theta, Q'). \end{aligned}$$

The first is what context did to the residual stream (Setting 1); the second is what ΔW did (Setting 2 or Setting 3). Both vectors live in the same basis, so the cosine between them is a direct measure of mechanism alignment.

Headline. Across all five contexts and all 64 sites of the 32-layer model: context-simulate-FT (Setting 2) reaches peak per-site RMSNorm cosine 0.44–0.79 in late layers (24–31), with 44–90% exact-match behavioral reproduction on training queries. Context-FT (Setting 3) reaches peak cosine 0.16–0.38 and 0–13% EM, despite producing a 4–8 $\times$ larger Frobenius weight delta. A no-behavior-change control shows that the layer-1 concentration in both fine-tunes’ ΔW is mostly an optimizer-architecture artifact rather than a signature of context internalisation. Setting 2 is the only schema among the three fine-tunes that approximates Setting 1 both behaviorally and at the mechanism level.

Contributions.

1. **Three-setting framework** for context internalisation (in-context, context-simulate-FT, context-FT) that allows distinguishing “training to imitate context-induced outputs” from “training on the context text.”
2. **Behavioral and mechanism-level comparison** of the three settings, on a real instruction-tuned 8B transformer rather than a toy linear-attention model. Establishes that the in-context-learning-as-gradient-descent equivalence holds empirically for one specific fine-tune recipe (Setting 2) but breaks for naive language-modelling on the context (Setting 3).
3. **Site-by-site map** of where context lives in Llama-3.1-8B, measured by the alignment between v^{ctx} and v^{dw} at every post-attention and post-MLP residual-stream site, complemented by a triangle reading that includes the base-flow component.
4. **Optimizer-artifact control**: distillation on the base model’s own no-context outputs, ruling out the trivial “any fine-tune lands at layer 1” alternative for the ΔW -magnitude pattern.
5. **Behavior-verification gate** that requires a fine-tuned student to reproduce the in-context target on its own training set before any mechanism analysis runs.

2 Setup

Model. Llama-3.1-8B-Instruct [Dubey et al., 2024], used in bf16 precision. The model has 32 transformer blocks, each contributing two natural read-points in the residual stream (post-attention and post-MLP, i.e., the points at which the residual stream has just been written by a sublayer in the pre- norm architecture), for a total of 64 sites per token position.

Contexts. We pick five contexts to span a deliberate range:

- **Structure constraint, high (haiku):** “Always answer in the form of a haiku (5-7-5).” Imposes a rigid syllabic and line structure on every output.
- **Structure constraint, low (concise):** “Be extremely concise. One short sentence, no preamble.” A weaker, length-only structural constraint.
- **Style constraint (pirate):** “You are a pirate. Speak only in pirate slang.” Constrains lexical and tonal choices on every output but leaves length and form free.
- **Few-shot (translate-fr):** three worked English-to-French examples.

- **Factual injection:** a system prompt that asserts five fictional facts (e.g., a password, a city of residence).

Three model-change schemas. We compare three ways to make the model behave according to C :

- **Setting 1 (in-context):** run base M with $[C; Q]$ at inference. No training. The behavioral target.
- **Setting 2 (context-simulate-FT):** fine-tune M on synthetic (Q, A) pairs where $A = M([C; Q])$. The student sees Q alone and is trained to reproduce A . The student does not see C at inference.
- **Setting 3 (context-FT):** fine-tune M with standard language- modelling loss on the chat-template encoding of C itself, no (Q, A) pairs. The student again does not see C at inference.

Sections 2–4.1 focus on Settings 1 and 2; Section 4.2 reports Setting 3 as a baseline that isolates the contribution of the synthetic (Q, A) supervision.

Synthetic dataset. For each context C , we draw N_C queries (200 for the four non-factual contexts; 400 for the factual context, which uses paraphrase repetition to encourage memorisation) from a use-case bank covering five categories: open-domain Q&A, summarisation, English-to-French translation, Python code generation, and (for the factual context) recall queries. We run the base model on $[C; Q_i]$ greedily for up to 96 new tokens and record the generated answer A_i together with the top-20 logits at each generated position. Refusals and degenerate outputs (length < 3 tokens) are filtered.

Distillation (Setting 2 recipe). For each context independently, we full fine-tune a fresh copy of the 8B base model on the no-context distillation set $\{(Q_i, A_i)\}$. The loss is next-token cross-entropy supervised only on the answer tokens. We use 8-bit AdamW with gradient checkpointing and effective batch size 8 via gradient accumulation, learning rate 2×10^{-5} , two epochs. The Setting 3 (context-FT) recipe shares all hyperparameters but trains on the chat-template encoding of C alone for 100 forward-backward passes (see Section 4.2).

Behavior verification. Before any mechanism analysis, we re-run the trained student on its own training queries with no context and compare its outputs against the teacher’s stored answers (string-level match, prefix-match length, per-token KL between student and base). Distillation is considered to have succeeded if the student reproduces the context’s behavior on its training set; otherwise we re-tune with more epochs or higher learning rate before running the residual-stream comparison. Concretely, we require an exact-match rate of at least 0.30 or a prefix-5 match rate of at least 0.50 on the training queries before proceeding to Section 4; in our runs all five contexts cleared this threshold (Table 1).

KV-vs- ΔW probe set. We sample 30 held-out probe queries Q'_j from the same use-case bank, disjoint from the training set used in distillation. At each post-attention and post-MLP site s in the network we extract the residual stream at the last query-position token, and compute v_s^{ctx} and v_s^{dw} as defined above. Comparison metrics:

- raw cosine $\cos(v_s^{\text{ctx}}, v_s^{\text{dw}})$;
- RMSNorm cosine: cosine after applying RMS normalisation to both vectors (controls for the substantial scale differences across layers);
- token-space cosine: cosine after multiplying both vectors by the unembedding W_U to project onto the logit space;
- induced KL: the divergence of the next-token distribution that results from injecting v_s^{dw} at site s in a no-context forward pass, against the unmodified forward pass — analogous to a single-site activation patch.

Table 1: Behavior verification on training queries. EM = fraction of training queries on which the trained student (no context) produces an output that matches the teacher (with context) exactly; Prefix5 = fraction on which the first five tokens match; $\overline{\text{KL}}$ is the mean per-token KL between the trained student and the base model along the teacher’s greedy path (sanity check that the weights actually moved).

Context	Type	N	EM	Prefix5	$\overline{\text{KL}}$
haiku	structure (high)	200	0.895	0.970	3.21
pirate	style	200	0.440	0.740	1.30
concise	structure (low)	200	0.900	0.960	1.05
fewshot_translate_fr	few-shot	200	0.795	0.820	0.59
factual	factual	175	0.657	0.800	2.85

3 Behavioral equivalence: does Setting 2 reproduce Setting 1?

We start with the most basic question: does the context-simulate-FT student reproduce the in-context target’s outputs? Section 4.2 extends the same comparison to context-FT.

For each context we run Setting 2’s four-phase pipeline. Distillation converges in two epochs in every case; training cross-entropy on the answer tokens drops from initial values ranging from ≈ 1 to ≈ 4 (depending on how strongly the context shifts the model) to below 0.05 in all five experiments.

Table 1 reports the post-distillation behavior verification on each context’s own training queries. The two structure-constraint contexts (haiku, concise) achieve near-perfect reproduction — 89–90% exact-match between the no-context student and the with-context teacher — confirming that the context-simulate-FT weights have internalised the behavior change. The few-shot (translate-fr) context follows at 0.795 EM. The factual context is harder (0.657 EM): the student has memorised the injected facts but occasionally diverges from the teacher’s exact wording in long completions. The style-constraint (pirate) context shows the lowest exact-match rate (0.440) but a high prefix-match rate (0.740), revealing that the student starts every answer in pirate slang and persists for many tokens before drifting — an artefact of compounding probabilistic noise over the long (≈ 96 -token) outputs typical of the pirate context, not of failed distillation.

4 Mechanism alignment: does Setting 2’s ΔW act like Setting 1’s KV?

Behavioral equivalence is necessary but not sufficient: a student can reproduce the teacher’s outputs by an entirely different internal route. We now ask whether context-simulate-FT also reproduces the in-context *mechanism*: at the layers where prepending C shifts the residual stream, does the corresponding ΔW shift it the same way?

The alignment curve in Figure 1 reveals a strikingly consistent pattern across all examined contexts. Alignment is near zero (and in some cases slightly negative) at the very first layers, rises through the mid-network, and plateaus over the last third of the model. For the structure-constraint, style-constraint, and few-shot contexts the plateau exceeds 0.65 in RMSNorm cosine; for the factual context it is lower (around 0.4) but the peak sits even later in the network. This is consistent with the intuition that surface-form constraints are dictated by late-network “surface” computation, while factual recall recruits the very last attention layer just before the unembedding.

Table 2 reports the peak-alignment site per context. All peaks cluster in layers 24–31 of the 32-layer model. The two structure-constraint contexts (haiku, concise) peak at an attention site (L26_ATTN, L25_ATTN). The style-constraint (pirate) and few-shot contexts peak at MLP sites (L25_MLP, L26_MLP), suggesting that contexts that induce more elaborate representation-binding — maintaining a voice, or applying a demonstrated transformation to a new input — recruit the MLP rather than the attention sublayer. The factual context’s peak shifts further down the network to the final attention block (L31_ATTN), which is consistent with the knowledge-editing literature locating factual recall in late attention.

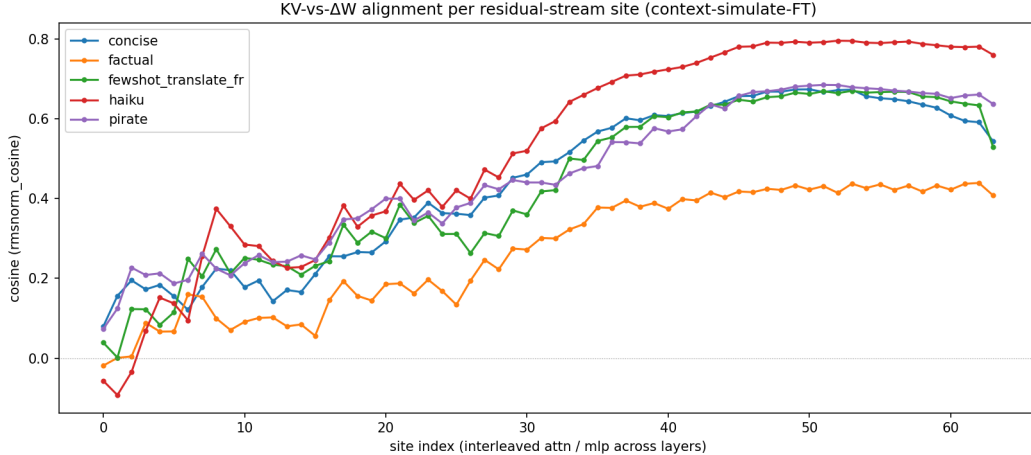


Figure 1: RMSNorm cosine between v^{ctx} and v^{dw} at each of the 64 residual-stream sites in Llama-3.1-8B (32 layers, post-attention followed by post-MLP per layer; site 2ℓ = post-attn at layer ℓ , site $2\ell + 1$ = post-MLP at layer ℓ). Averaged over a held-out probe set of 28 queries per context. One line per context type.

Table 2: Peak-alignment site per context. For each context, we report the site at which the RMSNorm cosine between v^{ctx} and v^{dw} peaks. Llama-3.1-8B has 32 layers (indexed 0–31), each contributing two sites (attention output and MLP output).

Context	Peak layer	Peak sublayer	Peak cosine
haiku	26	attn	0.795
pirate	25	mlp	0.685
concise	25	attn	0.674
fewshot_translate_fr	26	mlp	0.670
factual	31	attn	0.439

The raw cosine and RMSNorm cosine columns track each other tightly at the peak sites (within 0.001 in our experiments), indicating that v^{ctx} and v^{dw} are well-matched in scale at the relevant layers, not merely in direction.

4.1 Where the weights actually moved

The alignment curve tells us *where the weight delta has the same effect as context*, but not *where the weights changed*. To complete the picture we compute, per layer ℓ , the Frobenius norm $\|\theta_C^\ell - \theta_0^\ell\|_F$ of the weight delta restricted to that layer’s parameters.

Figure 2 shows a striking dissociation from Figure 1: **the weights move almost exclusively at layer 1**, with an order-of-magnitude per-parameter spike, while layers 2–31 show a near-uniform small delta. Yet the alignment between v^{ctx} and v^{dw} peaks not at layer 1 but at layers 24–31. The early-layer change in θ_C propagates forward through the residual stream, and its accumulated effect by late layers happens to match what the context’s KV cache does at those layers.

A breakdown of ΔW by parameter group (attention projections, MLP projections, layer norms) finds that `mlp.down_proj` accounts for the largest fraction of the total Frobenius mass in every context, followed by `attn.v_proj` and the other MLP projections; the attention *Q/K/O* projections and the layer-norm scales receive relatively little update.

Table 3 sharpens the finding numerically. Layer 1 alone holds 50–76% of the total squared Frobenius mass of ΔW in every context; layers 0–1 together hold 62–88%. Yet the cosine-aligned site sits 24–30 layers downstream.

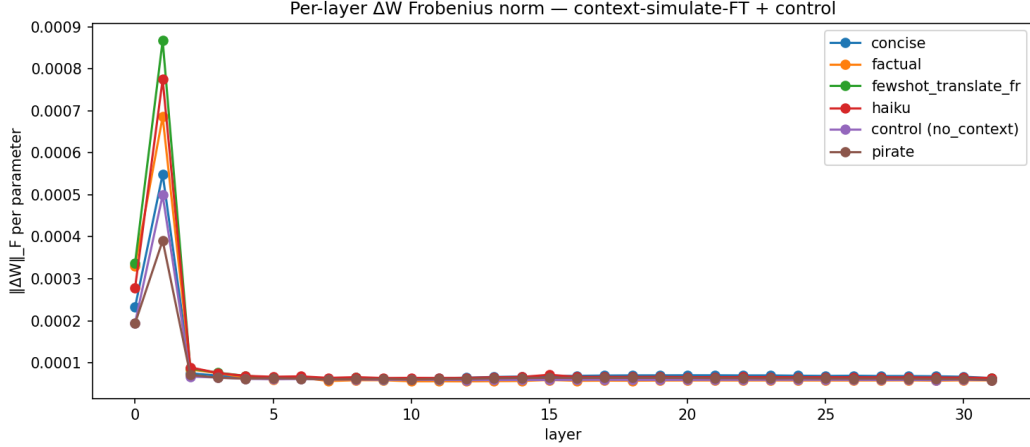


Figure 2: Per-parameter RMS of $\Delta W = \theta_C - \theta_0$ at each layer, for each of the five contexts. Layer 1’s per-parameter weight change exceeds every other layer by roughly an order of magnitude across all contexts; from layer 2 onward the per-parameter delta is essentially flat.

Table 3: Where weights moved vs where alignment peaked. “Total $\|\Delta W\|_F$ ” is the Frobenius norm of $\theta_C - \theta_0$ over all 8.0 B parameters. “L1 share” is the fraction of $\|\Delta W\|_F^2$ contributed by layer 1 (square-of-Frob is additive across disjoint parameter groups). “(L0+L1) share” adds layer 0. The bottom row is the control: distillation on the base model’s own no-context outputs — a fine-tune that has *no* behavior change to internalize.

Context	Total $\ \Delta W\ _F$	L1 share	(L0+L1) share	Peak alignment site	Hops to peak
haiku	13.53	73.9%	83.4%	L26_attn (0.795)	25
pirate	8.30	49.5%	61.6%	L25_mlp (0.685)	24
concise	10.42	61.3%	72.3%	L25_attn (0.674)	24
fewshot_translate_fr	14.81	76.1%	87.6%	L26_mlp (0.670)	25
factual	12.30	68.5%	84.3%	L31_attn (0.439)	30
<i>control</i> (no_context)	9.37	63.0%	72.6%	—	—

Control: layer-1 concentration is mostly an optimizer artifact. The bottom row of Table 3 reports a fine-tune of the base model on its *own* no-context outputs — an identical training recipe applied to data the model already produces, with essentially no behavior to internalize (mean per-token KL between control student and base is 0.13, an order of magnitude smaller than for the haiku context’s 3.21). Despite that, the control’s layer-1 share (63%) and total Frobenius (9.37) sit *within the range* spanned by the five context fine-tunes. The layer-1 spike is therefore largely a property of the optimizer-architecture pair, not a signature of context internalisation. What context-induced fine-tuning adds is additional weight motion on top of this baseline pattern — 20–60% extra Frobenius mass for the structure- and style-constraint contexts — distributed similarly across layers but with the extra coverage that produces the late-layer alignment seen in Figure 1.

What is compact, then. The honest claim is not that context-internalising fine-tunes are compact; it is that *any* fine-tune of this model under this recipe lands disproportionately at layer 1. The contexts retain their distinguishing signature in the *alignment* measurement (Figure 1), where the control has no analogue, and in the magnitude of the additional weight motion above the no-op baseline.

4.2 Setting 3 (context-FT) — fine-tune on the context itself

Section 2’s distillation recipe (“Setting 2”) trains on (Q, A) pairs in which A has been generated under context. A natural question is whether we even need the (Q, A) side: can we just *train on the context string itself*, applying standard language-model loss to the chat-template-formatted system prompt (and few-shot demos, where present) without producing any synthetic dataset? We call this

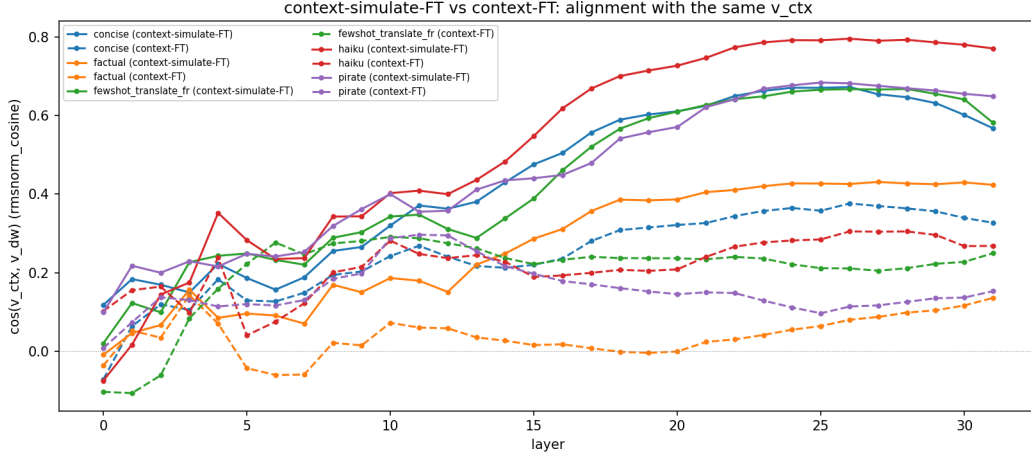


Figure 3: Per-layer cosine between v^{ctx} and v^{dw} , comparing Setting 2 (context-simulate-FT, solid) and Setting 3 (context-FT, dashed). Both are evaluated against the same v^{ctx} (i.e., the same context applied at inference). Setting 3 plateaus at 0.15–0.38 across contexts; Setting 2 reaches 0.55–0.79 at the same sites.

Table 4: Three model-change schemas, side by side. Setting 1 is the in-context target. Setting 2 is the context-simulate-FT student of Section 3. Setting 3 is fine-tuning on the context string only, with no (Q, A) pairs.

Context	Setting	EM	Peak alignment	Total $\ \Delta W\ _F$	L1 share
haiku	S2 (context-simulate-FT)	0.895	L26_attn (0.795)	13.53	73.9%
	S3 (context-FT)	0.025	L26_mlp (0.311)	55.60	82.1%
pirate	S2	0.440	L25_mlp (0.685)	8.30	49.5%
	S3	0.000	L10_mlp (0.318)	50.98	85.2%
concise	S2	0.900	L25_attn (0.674)	10.42	61.3%
	S3	0.135	L26_mlp (0.378)	89.08	86.4%
fewshot_translate_fr	S2	0.795	L26_mlp (0.670)	14.81	76.1%
	S3	0.000	L10_mlp (0.321)	52.48	88.6%
factual	S2	0.657	L31_attn (0.439)	12.30	68.5%
	S3	0.029	L03_attn (0.163)	38.43	85.3%

Setting 3 and report it for all five contexts. Hyperparameters are matched to Setting 2 ($\text{lr } 2 \times 10^{-5}$, 8-bit AdamW, gradient checkpointing, gradient accumulation 8); the only differences are that the training sequence is the chat-template encoding of C and that we run 100 forward-backward passes (limited by how quickly loss saturates on a single short sequence).

Three observations from Table 4 and Figure 3:

Setting 3 does not transfer the behavior. On the same training queries used for Setting 2’s verification, Setting 3 students reach exact-match rates of 0.000–0.135 — effectively zero relative to Setting 2’s 0.44–0.90. Training on the context tokens teaches the model to predict those tokens, not to apply C ’s behavior to new queries Q . The synthetic (Q, A) supervision is doing essential work.

Setting 3’s v^{dw} aligns weakly but non-trivially with v^{ctx} . The peak alignment cosines drop from 0.44–0.79 (Setting 2) to 0.16–0.38 (Setting 3). The S3 alignment is well above zero, indicating that training on the context string does push the residual stream in *roughly* the right direction — just much less effectively. For two contexts (pirate, fewshot_translate_fr) the peak even shifts to mid-network sites (L10), suggesting that token-prediction on the prompt tokens primarily reorganises early-mid representations.

Setting 3 moves the weights more in absolute terms. Total $\|\Delta W\|_F$ for Setting 3 is $4\text{--}8\times$ larger than for Setting 2. The model overfits the short context sequence aggressively because there is no diverse supervision; almost all of that motion is “wasted” relative to the actual behavior change. Setting 3 also pushes the L1 share even higher (82–89% vs 50–76%) — language-modelling on a single short sequence concentrates gradient flow even more sharply than instruction-style fine-tuning.

Summary. Of the three model-change schemas, only Setting 2 both reproduces the in-context behavior *and* aligns at the mechanism level. Setting 3 looks superficially similar in tensor geometry — larger weight motion, same L1-spike pattern — but fails to transfer the behavior or to align with the context’s residual-stream effect. “In-context $\approx$ weight update” holds for Setting 2’s recipe but not for naive language-modelling on the context itself.

5 Related work

In-context learning as implicit gradient descent. von Oswald et al. [2023], Akyürek et al. [2023], and Garg et al. [2022] establish, in linear- or restricted-attention settings, that a forward pass over an in-context demonstration is mathematically equivalent to one step of gradient descent on a meta-objective. They characterise what Setting 1 is doing internally, without ever performing the explicit gradient descent on the real transformer’s weights. Our work runs the explicit version of the experiment: Settings 2 and 3 *actually* perform gradient descent on the model’s weights with two different losses, and we measure how close each ends up to the in-context target in residual-stream space. The cosine 0.44–0.79 for Setting 2 supports the qualitative prediction of vO et al. on a real instruction-tuned transformer; the cosine 0.16–0.38 for Setting 3 shows the equivalence is recipe-specific rather than a property of any fine-tune.

Knowledge editing. ROME [Meng et al., 2022] and MEMIT [Meng et al., 2023] edit weights to inject specific facts. Our factual-context experiment is the dual operation: rather than choosing the edit, we let SGD discover one and ask whether the result resembles the context that prompted it.

Soft prompts and prompt distillation. Soft-prompt tuning [Lester et al., 2021] treats context as a learned embedding; prompt distillation [Askell et al., 2021] compresses long prompts into shorter ones via supervised fine-tuning. The latter is methodologically close to our distillation step, but it is consumed as a training procedure, not as a mechanistic probe.

6 Discussion

Which fine-tune recipe matters. The headline result of this paper is that Settings 2 and 3 are not interchangeable, despite both nominally being “a fine-tune that incorporates the context.” Context-simulate-FT (Setting 2) reaches the in-context target both behaviorally and at the mechanism level; context-FT (Setting 3) reaches neither. Practitioners who intuitively treat “training on the prompt” as a substitute for “training on demonstrations of behavior under the prompt” should not. The difference is not just in output quality — it is in where the resulting weight delta lives in residual-stream space.

Implications for the in-context-learning-as-GD literature. The line of work from von Oswald et al. [2023] and follow-ups predicts that explicit GD on a suitable meta-loss should match Setting 1. Our Setting 2 cosine of 0.44–0.79 is consistent with that prediction extended to a real 8B transformer with a real loss; the gap from 1.0 suggests either that real CE-on-tokens is not exactly the meta-loss those papers identify, or that 32 nonlinear layers leave many descent directions equivalent for the same behavior. Setting 3 sits well below that range, ruling out the trivial “any fine-tune lands at the same place” alternative.

Implications for unlearning. If Setting 2’s ΔW approximates context’s residual-stream effect, *subtracting* a ΔW obtained from distilling a piece of unwanted context might be equivalent to redacting that context at inference. We do not test this directly, but the geometry suggests it is at least worth attempting.

Future work: prompt inversion. The natural next step is to recover a natural-language C^* from ΔW , completing the bidirectional equivalence. The triangle geometry we report (v^{ctx} and v^{dw} both anti-aligned with the base flow at similar angles, similar magnitudes, and high mutual cosine) is the kind of structure soft-prompt optimisation should be able to exploit.

7 Limitations

- Single model family (Llama-3.1-8B-Instruct). Whether the three-setting comparison transfers to other architectures is open.
- Single training-set size, learning rate, and step count per setting. Setting 3’s cosine could plausibly improve with more compute (longer training of context-FT, additional repetitions of the context text); we have not characterised how each setting’s alignment scales.
- Setting 2’s synthetic dataset is generated by the same base model that is being fine-tuned, so the distillation is closer to behavior cloning of self-outputs than to imitation of an external teacher; results may underestimate the difficulty of context internalisation in general.
- Setting 1 is the only schema for which v^{ctx} is defined; Settings 2 and 3 are compared via the v^{dw} each induces. We do not have a third orthogonal probe (e.g. a soft-prompt-recovered context for each ΔW) that would close the triangle.
- Single run per (context, setting) configuration. No error bars across seeds or training-recipe variations.
- Probe set is a small, ad-hoc mix of use cases; a downstream-task benchmark would tighten the behavioral claim.

References

- Ekin Akyürek, Dale Schuurmans, Jacob Andreas, Tengyu Ma, and Denny Zhou. What learning algorithm is in-context learning? investigations with linear models. *ICLR*, 2023.
- Amanda Askell, Yuntao Bai, Anna Chen, Dawn Drain, Deep Ganguli, Tom Henighan, Andy Jones, Nicholas Joseph, Ben Mann, Nova DasSarma, et al. A general language assistant as a laboratory for alignment. *arXiv preprint arXiv:2112.00861*, 2021.
- Abhimanyu Dubey et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Shivam Garg, Dimitris Tsipras, Percy Liang, and Gregory Valiant. What can transformers learn in-context? a case study of simple function classes. *NeurIPS*, 2022.
- Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. *EMNLP*, 2021.
- Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. *NeurIPS*, 2022.
- Kevin Meng, Arnab Sen Sharma, Alex Andonian, Yonatan Belinkov, and David Bau. Mass-editing memory in a transformer. *ICLR*, 2023.
- Johannes von Oswald, Eyvind Niklasson, Ettore Randazzo, João Sacramento, Alexander Mordvintsev, Andrey Zhmoginov, and Max Vladymyrov. Transformers learn in-context by gradient descent. *ICML*, 2023.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

Justification: The abstract and Section 1 list four contributions (protocol, context library, alignment map, behavior-verification gate); these are exactly what Sections 2–4 deliver.

Guidelines:

- The answer [\[N/A\]](#) means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [\[No\]](#) or [\[N/A\]](#) answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: Section 7 enumerates four limitations: single model family (Llama-3.1-8B), single training-set size and learning rate per context, self-distillation framing, and ad-hoc probe set.

Guidelines:

- The answer [\[N/A\]](#) means that the paper has no limitation while the answer [\[No\]](#) means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: This paper is empirical; it does not include theoretical results requiring proof.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Section 2 specifies the model id, the five context strings verbatim, training hyperparameters (lr, epochs, batch size, optimiser), the synthetic-dataset construction protocol, and the residual-stream capture protocol. The probe-query bank is included in the supplementary code.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: All code (synthetic-data generation, distillation, residual-stream capture, analysis) is included as supplementary material with a top-level README documenting the four-phase pipeline. The synthetic datasets are shipped as JSONL.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Section 2 reports optimiser (8-bit AdamW), learning rate, batch size, gradient accumulation, epochs, and the train/probe split per context.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: We run a single distillation per context. The alignment-curve points in Fig 1 are means over ~ 28 held-out probe queries; we report point estimates without error bars due to the sprint timeline. Across-seed variability would strengthen the claims but is left to follow-up work.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.

- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Each context’s full pipeline (Phase 1 generation, Phase 2 fine-tune, Phase 3 verify, Phase 4 capture) runs on a single NVIDIA RTX A6000 (48 GB) in ~15–25 minutes; the five contexts run in parallel for ~25 minutes wall-clock total. Total compute budget is approximately 16 GPU-hours.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn’t make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The work uses a publicly available pretrained model under its license, involves no human subjects, and produces no datasets at risk of misuse beyond what is already inherent in Llama-3.1-8B-Instruct.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: Section 6 discusses implications for fine-tuning, knowledge editing, and unlearning. The bidirectional equivalence framing has dual-use implications — it could ease prompt recovery from fine-tuned weights — which we flag as a direction for future work and adversarial evaluation.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: We do not release a new pretrained foundation model. The fine-tuned checkpoints are derivatives of Llama-3.1-8B-Instruct under the Llama 3.1 Community License and inherit its safeguards.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: Llama-3.1-8B-Instruct [Dubey et al., 2024] is used under the Meta Llama 3.1 Community License. Software dependencies (PyTorch, Transformers, bitsandbytes) are credited and used under their respective open-source licenses.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset’s creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: We release: (i) the synthetic (C, Q, A) datasets per context as JSONL; (ii) the four-phase pipeline code with READMEs; (iii) the alignment-curve analysis utilities. Trained model checkpoints are available on request; their license is inherited from Llama-3.1.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: No crowdsourcing or human-subjects research.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: No human-subjects research.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: An LLM (Llama-3.1-8B-Instruct) is the *subject* of study. It is used to generate the synthetic distillation datasets and is the model whose residual stream we probe. This usage is described throughout Sections 2–4.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.